

SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)
Department of Computer Science and Engineering

ASSESSMENT TOOL 2 - FLOWCHART-TO-CODE CONVERSION

Course Code:	CSA0305	Course Title:	Data Structures
CO Assessed:	CO5 - Naturalization (BL4, BL6)	Assessment:	Assessment Tool 2 - Flowchart-To-Code Conversion
Weightage:	35% of CIA	Total Marks:	100
CO5 Statement:	Construct MST using shortest path algorithms and traverse the graph to model and analyse real-world problem.		
Student Name:	Shahana	Reg. No.:	192524479
Section / Batch:	Slot A	Date:	04/09/2026

Code of Conduct

I Shahana (192524479)

(Name / Reg No)

certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: Shahana

Instructions

- This test contains 5 Flowchart-to-code conversion questions. Total: 100 Marks.
- When a student answer using Flowchart-to-code conversion should evaluate based on the ability to design clear, logical, and efficient code solutions for data structure problems.
- No negative marking. Unanswered questions carry zero marks.

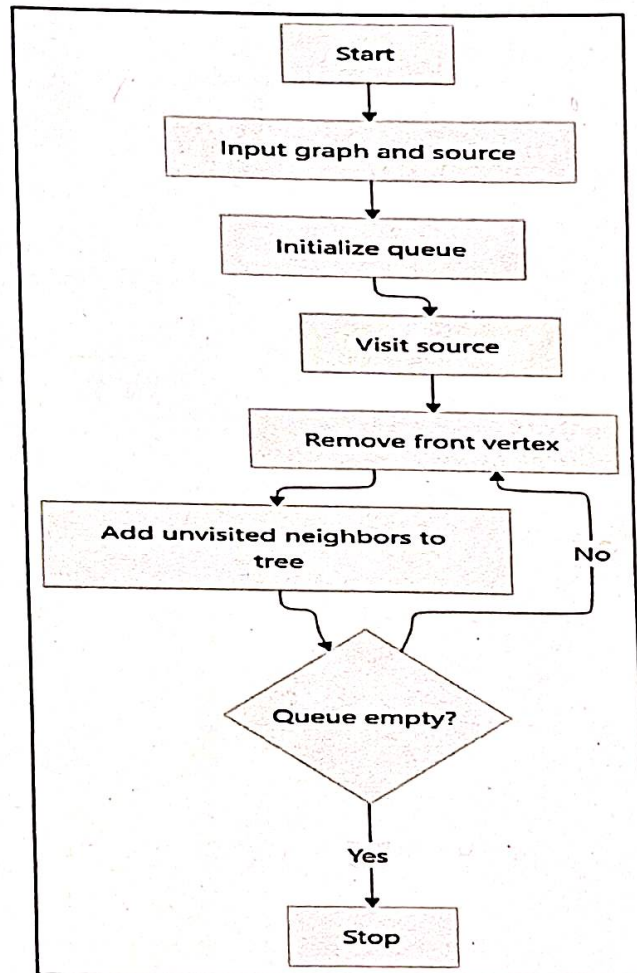
Section / Topic	Focus	Q Nos	Marks
Graph Traversal	BFS	1	20
Shortest Path Algorithm	Dijkstra's Algorithm	2	20
Shortest Path Algorithm	Unweighted Graph	3	20
Graph Traversal	BFS	4	20
Minimum Spanning Tree	Prim's Algorithm	5	20
GRAND TOTAL:			100 Marks

Annexure A - Pseudocode Writing Task

1. Convert the flowchart for generating a BFS tree into code.

(20 Marks)

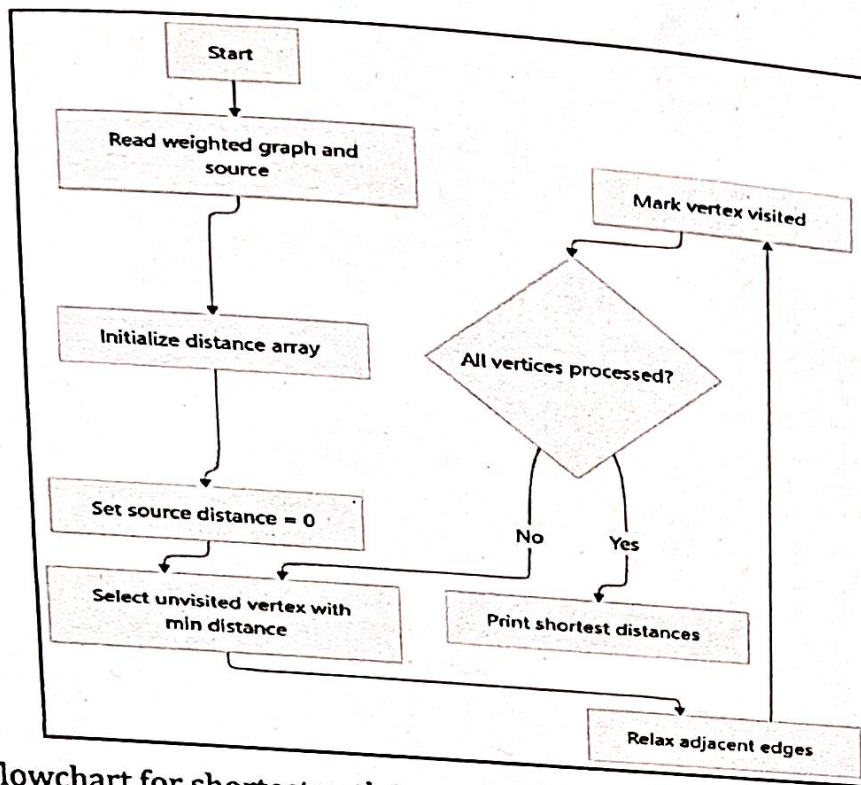
Flowchart Diagram:



2. Convert the flowchart for Dijkstra's shortest path algorithm into code.

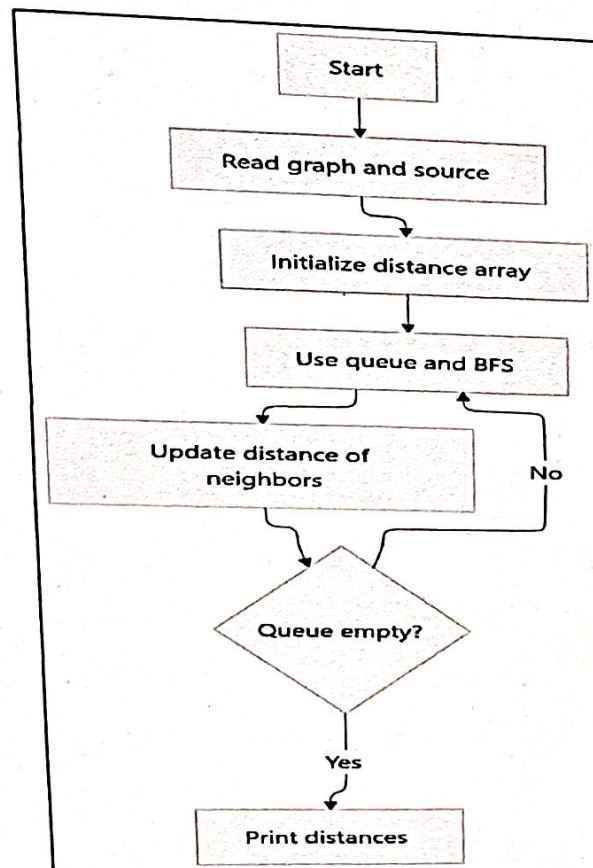
(20 Marks)

Flowchart Diagram:



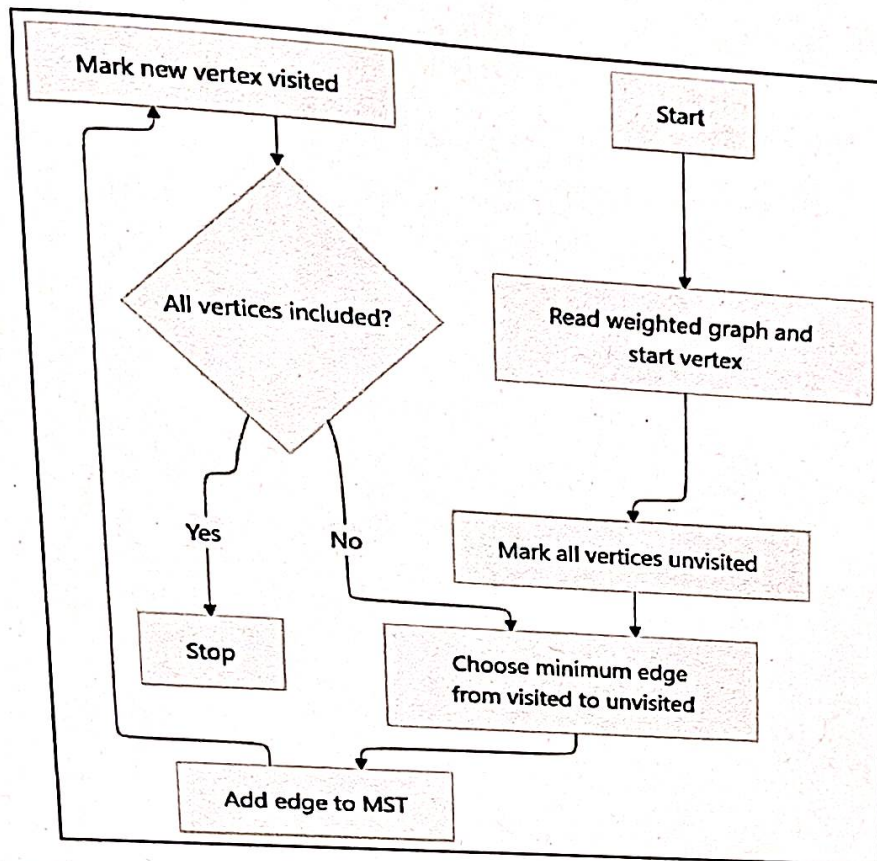
3. Convert the flowchart for shortest path in an unweighted graph into code.
Flowchart Diagram:

(20 Marks)



4. Convert the flowchart for Prim's algorithm into code.
Flowchart Diagram:

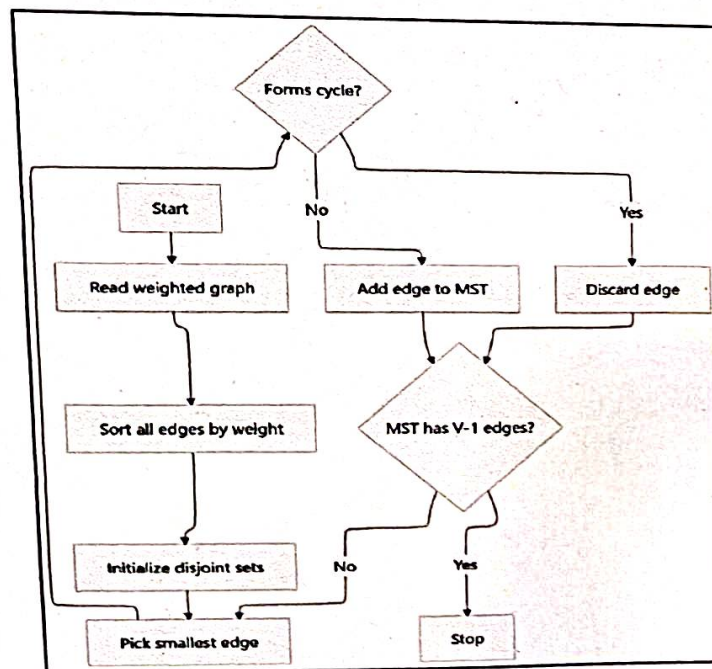
(20 Marks)



5. Convert the flowchart for Kruskal's algorithm into code.

(20 Marks)

Flowchart Diagram:



rubrics for Calculation

Criteria	Weightage	Level 4 - Exemplary	Level 3 - Proficient	Level 2 - Developing	Level 1 - Beginning
Problem Understanding	20	Demonstrates complete and clear understanding; handles edge cases effectively	Good understanding; minor gaps in edge case handling	Partial understanding; misses some requirements	Misinterprets problem; major gaps
Code Correctness	30	Fully correct; passes all test cases	Mostly correct; minor errors	Partially correct; several test cases fail	Incorrect or non-functional code
Code Quality & Readability	20	Clean, modular, well-commented, follows standards	Readable with some structure	Limited readability; poor structure	Unorganized and hard to read
Complexity Analysis	20	Accurate time & space complexity with justification	Correct but limited explanation	Incomplete or partially correct	Missing or incorrect
Testing & Validation	10	Thorough testing including edge cases	Adequate testing with few edge cases	Minimal testing	No testing evidence

Faculty In-charge	Course Coordinator	Program Director
Signature: 	Signature: _____	Signature: _____
Date: _____	Date: _____	Date: _____

BFS (Breadth First Search)

```
#include <iostream>
```

```
#include <queue>
```

```
using namespace std;
```

```
int main()
```

```
{
```

```
    int n, start;
```

```
    int a[10][10];
```

```
    cout << "Enter number of vertices: ";
```

```
    cin >> n;
```

```
    cout << "Enter number of vertices";
```

```
    cin >> n;
```

```
    cout << "Enter adjacency matrix: \n";
```

```
    for (int i=0; i<n; i++)
```

```
        for (int j=0; j<n; j++)
```

```
            cin >> a[i][j];
```

```
    cout << "Enter starting vertex: ";
```

```
    cin >> start;
```

```
    bool visited[10] = {false};
```

```
    queue<int> q;
```

```
    visited[start] = true;
```

```
    q.push(start);
```

```
    cout << "BFS:";
```

```
    while (!q.empty())
```

```
    { int v = q.front();
```

```
      q.pop();
```

```
      cout << v << " ";
```



```

for (int i = 0; i < n; i++)
{
    if (q[u][i] == 1 && !visited[i])
    {
        visited[i] = true;
        q.push(i);
    }
}
return 0;
}

```

2. Dijkstra's Shortest Path.

```

#include <iostream>
using namespace std;
int main()
{
    int n, start;
    int a[10][10];
    cout << "Enter number of vertices: ";
    cin >> n;
    cout << "Enter cost matrix: \n";
    for (int i = 0; i < n; i++)
        for (int j = 0; j < n; j++)
            cin >> a[i][j];

    cout << "Enter starting vertex: ";
    cin >> start;
    int dist[10];
    bool visited[10] = {false};
    for (int i = 0; i < n; i++)
        dist[i] = 9999;
    dist[start] = 0;
}

```

```

    countZn; count++)
    int u = -1;
    for (int i = 0; i < n; i++)
    {
        if (!visited[i] && (u == -1 || dist[i] < dist[u]))
            u = i;
    }
    visited[u] = true;
    for (int v = 0; v < n; v++)
    {
        if (a[u][v] != 0 && dist[u] + a[u][v] < dist[v])
            dist[v] = dist[u] + a[u][v];
    }
}

cout << "Shortest distances: \n";
for (int i = 0; i < n; i++)
{
    cout << start << " - -> " << i << " = " << dist[i] << endl;
}
return 0;
}

```

3. Shortest Path in an Unweighted Graph.

```

#include <iostream>
#include <queue>
using namespace std;
int main()
{
    int n, start;
    int a[10][10];
    cout << "Enter number of vertices: ";
}

```



```
cin >> n;
```

```
cout << "enter adjacency matrix: \n";
```

```
for (int i=0; i<n; i++)
```

```
    for (int j=0; j<n; j++)
```

```
        cin >> a[i][j];
```

```
cout << "Enter starting vertex: ";
```

```
cin >> start;
```

```
int dist[n];
```

```
bool visited[n] = {false};
```

```
for (int i=0; i<n; i++)
```

```
    dist[i] = -1;
```

```
queue<int> q;
```

```
visited[start] = true;
```

```
dist[start] = 0;
```

```
q.push(start);
```

```
while (!q.empty())
```

```
{ int u = q.front();
```

```
  q.pop();
```

```
  for (int v=0; v<n; v++)
```

```
      if (a[u][v] == 1 && !visited[v])
```

```
      { visited[v] = true;
```

```
        dist[v] = dist[u] + 1;
```

```
        q.push(v);
```

```
      }
```

```
  }
```

```
  cout << "Start << " << " << " << dist[i] << endl;
```

```
return 0;
```

```
#include <iostream>
```

```
using namespace std;
```

```
int main() {
```

```
int V=5;
```

```
vector <vector<int>> g = {
```

```
{0,2,0,6,0}, {2,0,3,8,5}, {0,3,0,0,7}, {6,8,0,0,9},  
{0,5,7,9,0}
```

```
};
```

```
vector<int> key(V, INT_MAX), parent(V, -1);
```

```
vector<bool> vis(V, false);
```

```
key[0]=0;
```

```
for (int c=0; c<V-1; c++) {
```

```
int u=-1;
```

```
for (int v=0; v<V; v++)
```

```
if (!vis[v] && (u == -1 || key[v] < key[u])) u = v;
```

```
vis[u] = true;
```

```
for (int v=0; v<V; v++)
```

```
if (vis[v] && !vis[v] && g[u][v] < key[v])
```

```
key[v] = g[u][v], parent[v] = u;
```

```
}  
int total=0;
```

```
for (int i=1; i<V; i++) {
```

```
cout << parent[i] << " - " << i << " wt: " << g[i][parent[i]] << " \n";
```

```
total += g[i][parent[i]];
```

```
}
```

```
cout << " Total MST Weight: " << total;
```

```
}
```


5. Kruskal's Algorithm

```
#include <iostream>
```

```
using namespace std;
```

```
int par[5]
```

```
int find(int x) { return par[x] == x ? x : par[x] = find(par[x]); }
```

```
int main () {
```

```
    int v=5;
```

```
    vector<array<int,3>> e = {
```

```
        {2,0,1}, {6,0,3}, {3,1,2}, {8,1,3}, {5,1,4}, {7,2,4}, {9,3,4}
```

```
    };
```

```
    sort(e.begin(), e.end());
```

```
    for (int i=0; i<v; i++) par[i]=i;
```

```
    int total=0; count=0;
```

```
    for (auto& [w, u, v] : e) {
```

```
        if (count == v-1) break;
```

```
        int ru = find(u), rv = find(v);
```

```
        if (ru == rv) continue;
```

```
        par[ru] = rv;
```

```
        cout << u << " - " << v << " wt: " << w << "\n";
```

```
        total += w;
```

```
        count++;
```

```
    }
```

```
    cout << "Total MST weight : " << total;
```

```
    return 0;
```